

混合检索 RAG：多路召回 + 重排模型 已付费

原创 神说要有光 神光的幸福生活 2026年5月2日 21:07 山东

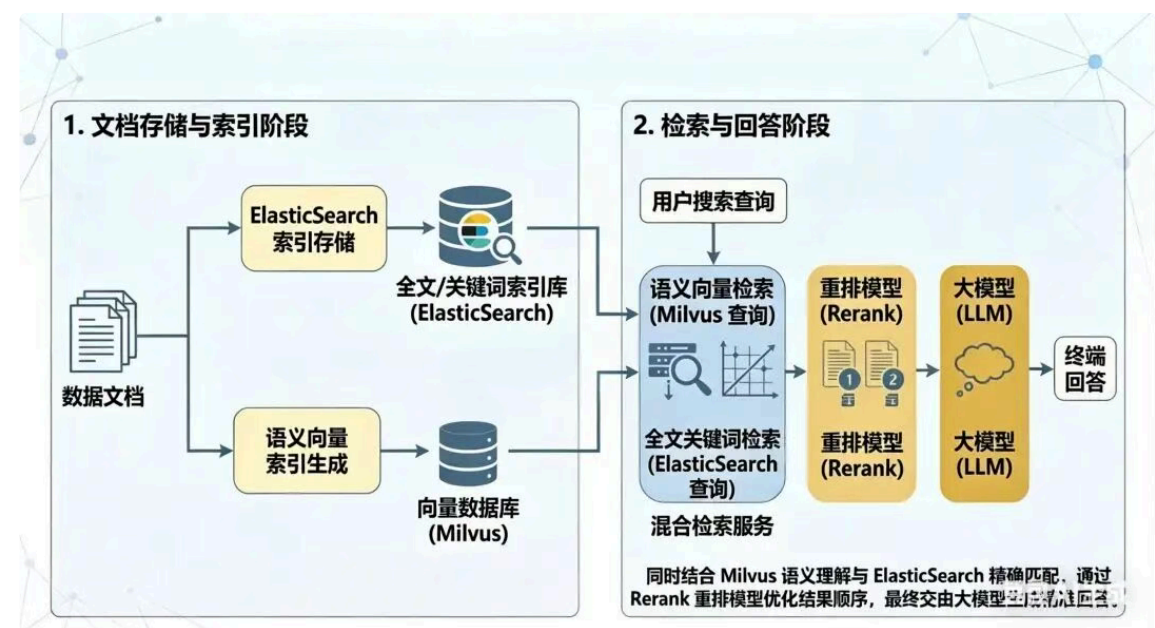
我们学了 Elasticsearch 的倒排索引的检索架构，实现了中文分词和海量文档的全文检索。

实际上，RAG 的语义检索有时候不好用：

- 专业术语、精确实体更适合关键词检索，纯语义检索容易匹配不准

所以我们会同时结合 Elasticsearch 关键词检索，Milvus 语义检索

然后查到的结果用 rerank 重排模型排序后给到大模型，来生成最终回答



也就是说，基于 Elasticsearch 的全文检索是 Agentic RAG 的混合检索架构中必备的一环。

我们先用代码实现下 ES 的文档的 CRUD：

在 es-test 里添加 src/create.mjs

```
import { Client } from '@elastic/elasticsearch';

const client = new Client({
  node: 'http://localhost:9200'
});

const INDEX_NAME = 'travel_journal';

asyncfunction createIndex() {
```

```
const exists = await client.indices.exists({ index: INDEX_NAME });
if (exists) {
  console.log(`❗ 索引已存在: ${INDEX_NAME}`);
  return;
}

await client.indices.create({
  index: INDEX_NAME,
  mappings: {
    properties: {
      note_title: { type: 'text', analyzer: 'ik_max_word', search_analyzer: 'ik_smart' },
      note_body: { type: 'text', analyzer: 'ik_max_word', search_analyzer: 'ik_smart' },
      tags: { type: 'keyword' },
      mood: { type: 'keyword' },
      priority: { type: 'integer' },
      created_at: { type: 'date' },
      updated_at: { type: 'date' }
    }
  }
});
```

```
console.log(`✅ 索引创建成功: ${INDEX_NAME}`);
}
```

```
asyncfunction seedData() {
  const now = new Date().toISOString();
  const docs = [
    {
      note_title: '杭州西湖半日游',
      note_body: '早上绕湖慢跑，中午吃片儿川，下午在断桥拍照放松。',
      tags: ['旅行', '周末', '杭州'],
      mood: 'relaxed',
      priority: 2,
      created_at: now,
      updated_at: now
    },
    {
      note_title: '城市骑行计划',
      note_body: '周六沿江骑行 20 公里，带上水和简易修车工具。',
      tags: ['运动', '骑行'],
      mood: 'energetic',
      priority: 3,
      created_at: now,
      updated_at: now
    },
    {
      note_title: '雨天宅家阅读',

```

```

      note_body: '下雨天在家看书，整理本周笔记并做晚餐。',
      tags: ['生活', '阅读'],
      mood: 'calm',
      priority: 1,
      created_at: now,
      updated_at: now
    }
  ];

const operations = docs.flatMap((doc) => [{ index: { _index: INDEX_NAME } }, doc]);
await client.bulk({ refresh: true, operations });
console.log(`✅ 初始化数据完成，共 ${docs.length} 条`);
}

asyncfunction run() {
  await createIndex();
  await seedData();
}

run().catch((err) => {
  console.error('❌ 创建阶段失败:', err);
  process.exit(1);
});

```

用 @elastic/elasticsearch 这个包来做一下索引的创建，数据的批量插入

安装依赖：

```
pnpm install @elastic/elasticsearch
```

创建好索引后，再来做一下增删改查：

src/operate.mjs

```
import { Client } from '@elastic/elasticsearch';

const client = new Client({
  node: 'http://localhost:9200'
});

const INDEX_NAME = 'travel_journal';

asyncfunction createDocument() {
  const now = new Date().toISOString();
  const res = await client.index({
    index: INDEX_NAME,
    document: {
      note_title: '夜跑复盘',
      note_body: '今天夜跑 5 公里，配速稳定，结束后做了拉伸。',
      tags: ['运动', '夜跑'],
      mood: 'focused',
      priority: 2,
      created_at: now,
      updated_at: now
    },
    refresh: true
  });

  console.log('✅ 新增成功, ID =', res._id);
  return res._id;
}

asyncfunction getDocument(docId) {
  const res = await client.get({
    index: INDEX_NAME,
    id: docId
  });
  console.log('🔍 查询结果:', res._source);
}

asyncfunction updateDocument(docId) {
  await client.update({
    index: INDEX_NAME,
```

```

    id: docId,
    doc: {
      note_body: '今天夜跑 6 公里, 状态不错, 拉伸后恢复很快。',
      tags: ['运动', '夜跑', '训练'],
      updated_at: new Date().toISOString()
    },
    refresh: true
  });
console.log('🔄 更新成功');
}

```

```

asyncfunction searchDocuments() {
const res = await client.search({
  index: INDEX_NAME,
  query: {
    match: {
      note_body: {
        query: '夜跑 训练',
        analyzer: 'ik_smart'
      }
    }
  }
});
}

```

```

const rows = res.hits.hits.map((item) => ({
  id: item._id,
  ...item._source
}));
console.log('🔍 搜索结果:', rows);
}

```

```

asyncfunction deleteDocument(docId) {
await client.delete({
  index: INDEX_NAME,
  id: docId,
  refresh: true
});
console.log('🗑️ 删除成功');
}

```

```

asyncfunction run() {
const docId = await createDocument();
await getDocument(docId);
await updateDocument(docId);
await searchDocuments();
await deleteDocument(docId);
}

```

```
run().catch((err) => {  
  console.error('❌ 操作阶段失败:', err);  
  process.exit(1);  
});
```

神光的编程秘籍

这样，代码里做索引的创建、文档的增删改查就完成了。

我们可以把一段 query 基于向量相似度检索出一些文档，分词后基于关键词检索出一些文档，然后把这些给到大模型来生成回答。

但是，全部给到大模型会有特别多无关内容，我们需要做一下筛选。

先把混合召回的文档做一次**重排序（rerank）**，把最相关、最有用、最能支撑回答的文档筛选出来，再丢给大模型生成最终答案。

为什么要做这一步？

- 混合召回（向量 + 关键词）会带来大量冗余信息
- 大模型上下文窗口有限，不能把所有文档都塞进去
- 噪声太多会让模型答非所问、逻辑混乱、幻觉增加
- 先过滤、再精简，才能让回答更精准

所以我们的流程会变成：

- 用户输入 query

- 多路召回
 - 向量检索（语义相似）
 - 关键词检索（分词匹配）
- 结果合并 → 去重
- 重排序 / 相关性打分（Rerank）
 - 把最相关的排在最前面
 - 只保留前 N 条最有用的文档
- 把高质量文档送入大模型
- 大模型基于干净上下文生成最终回答

这样做出来的 RAG 系统，才真正具备：

既能找得全、又能找得准，过滤掉无关杂音，减少大模型瞎编，生产用完全没问题。

按照这个思路，来改造下我们的 Agentic RAG。

重排模型也是专门的一种模型，就像嵌入模型是专用模型一样。

嵌入模型是把文本转成向量

重排模型是输入用户问题 + 一段文档，输出一个相关度分数

专门用来给 RAG 做去噪、筛选、重新排顺序，体量小、推理快、成本极低

那我们现在就把重排模型集成到我们的 Agentic RAG 里面。

先测一下重排模型的效果：

找一个重排模型：

<https://bailian.console.aliyun.com/cn-beijing?tab=model#/model-market>

神光的编程秘籍

curl 里测了之后，我们在代码里调用下

安装下 langchain 的包：

```
pnpm install @langchain/core @langchain/openai @langchain/langgraph @langchain/community
```

创建 .env

```
OPENAI_API_KEY=sk-xx
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
RERANK_URL=https://dashscope.aliyuncs.com/api/v1/services/rerank/text-rerank/text-rerank
MODEL_NAME=qwen-plus
RERANK_MODEL=qwen3-rerank
```

创建 src/rerank/dashscope-rerank.mjs

```
import "dotenv/config";
import { BaseDocumentCompressor } from "@langchain/core/retrievers/document_compressors";

export class DashScopeRerank extends BaseDocumentCompressor {

  constructor({ apiKey, model = "qwen3-rerank", topN = 3, baseUrl } = {}) {
    super();
    this.apiKey = apiKey;
    this.model = model;
    this.topN = topN;
    this.baseUrl = baseUrl ?? process.env.RERANK_URL;
  }

  async compressDocuments(documents, query, _callbacks) {
    const res = await fetch(this.baseUrl, {
      method: "POST",
      headers: {
        Authorization: `Bearer ${this.apiKey}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
```



```

        model: this.model,
        input: {
            query,
            documents: documents.map((d) => d.pageContent),
        },
        parameters: {
            return_documents: false,
            top_n: this.topN,
        },
    }),
});

const json = await res.json();
if (!res.ok) {
    throw new Error(
        `DashScope rerank ${res.status}: ${JSON.stringify(json)}`,
    );
}

const results = json?.output?.results;
if (!Array.isArray(results)) {
    throw new Error(`unexpected rerank response: ${JSON.stringify(json)}`);
}

return results.map((item) => documents[item.index]);
}
}

```

langchain 提供了 rerank 模型的基类 BaseDocumentCompressor，但是没有 qwen 重排模型对应的封装，我们自己封装下

和之前 curl 一样，调用接口，传入 query 和 documents，拿到重排序后的文档

然后调用下：

src/rerank/test.mjs

```

import "dotenv/config";
import { Document } from "@langchain/core/documents";
import { DashScopeRerank } from "../dashscope-rerank.mjs";

asyncfunction main() {
    const apiKey = process.env.OPENAI_API_KEY;

    const compressor = new DashScopeRerank({ apiKey, topN: 3 });

```

```

const query = "什么是文本排序模型";
const docs = [
  new Document({
    pageContent:
      "预训练语言模型的发展给文本排序模型带来了新的进展",
  }),
  new Document({
    pageContent: "量子计算是计算科学的一个前沿领域",
  }),
  new Document({
    pageContent: "文本排序模型广泛用于搜索引擎和推荐系统中...",
  }),
];

const ranked = await compressor.compressDocuments(docs, query);
console.log("重排后顺序 (pageContent): ");
for (const d of ranked) {
  console.log("-", d.pageContent);
}
}

main()

```

神光的编程秘籍

代码里调用 es 做关键词搜索，调用 rerank 模型来做问题和文档相关性排序都实现了

然后修改我们之前的检索逻辑：

- ES 关键词召回一批
- Milvus 向量召回一批
- 合并去重
- 丢给重排模型打分排序
- 只取前几条最相关的

检索之前要做一下数据写入，同样的数据分别写入 Milvus 和 ElasticSearch

首先改下 docker-compose.yml 把 milvus 加进去：

```
services:
  # Elasticsearch 8.17.0 + IK 中文分词 (内置到镜像)
  es:
    build: ./elasticsearch      # 从本地 Dockerfile 构建镜像 (自带IK)
    container_name: es-dev
    ports:
      - "9200:9200"            # ES 访问端口
    environment:
      - discovery.type=single-node # 单节点运行 (开发环境)
      - xpack.security.enabled=false # 关闭安全认证, 免密码访问
      - xpack.security.http.ssl.enabled=false # 关闭 HTTPS 加密
      - xpack.security.transport.ssl.enabled=false # 关闭节点传输加密
      - ES_JAVA_OPTS=-Xms512m -Xmx512m # JVM 内存配置, 避免占用过高
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/es/data:/usr/share/elasticsearch/data
    restart: always

  # Kibana 最新稳定版: 8.17.0 (必须与 ES 版本完全一致)
  kibana:
    image: kibana:8.17.0
    container_name: kibana-dev
    ports:
      - "5601:5601" # Kibana 网页控制台端口
    environment:
      - ELASTICSEARCH_HOSTS=http://es:9200 # 连接 ES 容器内部地址
    volumes:
      - ${DOCKER_VOLUME_DIRECTORY:-.}/volumes/kibana:/usr/share/kibana/data
    restart: always
    depends_on:
      - es # 等待 ES 启动完成后再启动 Kibana

  # Milvus 最新稳定版: 2.5.0 (必须与 ES 版本完全一致) # Milvus
  etcd:
    container_name: etcd-dev
    image: quay.io/coreos/etcd:v3.5.18
    environment:
```

- ETCD_AUTO_COMPACTION_MODE=revision
- ETCD_AUTO_COMPACTION_RETENTION=1000
- ETCD_QUOTA_BACKEND_BYTES=4294967296
- ETCD_SNAPSHOT_COUNT=50000

volumes:

- \${DOCKER_VOLUME_DIRECTORY:-.}/volumes/etcd:/etcd

command: etcd -advertise-client-urls=http://etcd:2379 -listen-client-urls http://0.0.

healthcheck:

```
test: ["CMD", "etcdctl", "endpoint", "health"]
interval: 30s
timeout: 20s
retries: 3
```

minio:

container_name: minio-dev

image: minio/minio:RELEASE.2024-05-28T17-19-04Z

environment:

```
MINIO_ACCESS_KEY: minioadmin
MINIO_SECRET_KEY: minioadmin
```

ports:

- "9001:9001"
- "9000:9000"

volumes:

- \${DOCKER_VOLUME_DIRECTORY:-.}/volumes/minio:/minio_data

command: minio server /minio_data --console-address ":9001"

healthcheck:

```
test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
interval: 30s
timeout: 20s
retries: 3
```

standalone:

container_name: standalone-dev

image: milvusdb/milvus:v2.5.25

command: ["milvus", "run", "standalone"]

security_opt:

- seccomp:unconfined

environment:

```
MINIO_REGION: us-east-1
ETCD_ENDPOINTS: etcd:2379
MINIO_ADDRESS: minio:9000
```

volumes:

- \${DOCKER_VOLUME_DIRECTORY:-.}/volumes/milvus:/var/lib/milvus

healthcheck:

```
test: ["CMD", "curl", "-f", "http://localhost:9091/healthz"]
interval: 30s
start_period: 90s
timeout: 20s
```

```
    retries: 3
  ports:
    - "19530:19530"
    - "9091:9091"
  depends_on:
    - "etcd"
    - "minio"

networks:
  default:
    name: common-network
```

神光的编程秘籍

然后做一下数据写入

创建 src/rag/seed-data.mjs

```
/**
 * ES + Milvus 种子数据。
 */
import 'dotenv/config';
import { Client } from '@elastic/elasticsearch';
import { OpenAIEmbeddings } from '@langchain/openai';
import {
  DataType,
  IndexType,
  MetricType,
```

```
MilvusClient,
} from '@zilliz/milvus2-sdk-node';

const INDEX_NAME = 'life_notes';
const ES_NODE = 'http://localhost:9200';
const MILVUS_ADDRESS = 'localhost:19530';

const DOC_TEXT = 'doc_text';
const EMBEDDING = 'embedding';

const ROWS = [
  {
    id: 'life_01',
    note_title: '周末煲汤小备忘',
    note_body:
      '排骨冷水下锅焯一下，加姜片料酒；换了砂锅小火炖一小时，最后放盐和白胡椒，海带要提前泡发切条。',
    tags: ['下厨', '周末'],
    mood: '馋',
    priority: 2,
  },
  {
    id: 'life_02',
    note_title: '晚饭后遛狗路线',
    note_body:
      '小区东门出去沿河岸走一圈大概四十分钟，记得带拾便袋和水壶；下雨天改地下停车场那层绕两圈也行。',
    tags: ['宠物', '散步'],
    mood: '放松',
    priority: 3,
  },
  {
    id: 'life_03',
    note_title: '阳台绿植浇水频率',
    note_body:
      '绿萝见干再浇，龟背竹叶面可以偶尔喷水；夏天蒸发快早上看一眼土表，冬天少浇防止烂根。',
    tags: ['家务', '植物'],
    mood: '碎碎念',
    priority: 1,
  },
  {
    id: 'life_04',
    note_title: '路由器偶尔断流排查笔记',
    note_body:
      '先重启光猫再重启路由；信道改成自动或固定 36；固件升级到官网最新版；还不行就还原出厂单独测网线。',
    tags: ['数码', '折腾'],
    mood: '烦躁',
    priority: 2,
  },
],
```

```
{
  id: 'life_05',
  note_title: '净水器滤芯更换记录',
  note_body:
    '官网登记的机身序列 SN-MILO-77821; 上次换的是第三代 RO 复合滤芯, 配件订单号 PO-20250409-K9; ',
  tags: ['家务', '维保'],
  mood: '琐事',
  priority: 1,
},
{
  id: 'life_06',
  note_title: '梧州龟苓膏粉冲泡比例',
  note_body:
    '双钱牌粉一包兑常温凉水先搅匀再小火搅拌到冒小泡; 千万别用滚烫开水直接冲容易结块; 可加少量桂花蜜。',
  tags: ['下厨', '甜品'],
  mood: '解馋',
  priority: 1,
},
{
  id: 'life_07',
  note_title: '租房合同划的重点句',
  note_body:
    '第八条写的是押一付三提前三十日书面通知; 手写补充了一句「甲方不得以不正当理由扣减退房押金」记得双方i
  tags: ['租房', '法律'],
  mood: '谨慎',
  priority: 3,
},
{
  id: 'life_08',
  note_title: '肉汤熬久了反而涩',
  note_body:
    '大块骨肉要先焯掉浮沫, 文火咕嘟太久胶质出来了汤会发黏发涩; 觉得不爽可以中途打掉一层油, 起锅前再调l
  tags: ['下厨', '技巧'],
  mood: '琢磨',
  priority: 2,
},
{
  id: 'life_09',
  note_title: '半夜趴窗台透气',
  note_body:
    '脑子停不下来就一直复盘白天在会上说的话, 越想越清醒; 干脆开窗吹两分钟冷风, 把手机扔到客厅充电再回屋,
  tags: ['情绪', '失眠'],
  mood: '飘',
  priority: 2,
},
{
  id: 'life_10',
```

```

    note_title: '出差酒店网速玄学',
    note_body:
      '同一个SSID走廊尽头满格会议室里假信号；连手机热点写周报反而稳；视频会议尽量靠窗座位别躲在最里间死角
    tags: ['差旅', '办公'],
    mood: '无奈',
    priority: 2,
  },
];

const embeddings = new OpenAIEmbeddings({
  apiKey: process.env.OPENAI_API_KEY,
  model: process.env.EMBEDDINGS_MODEL_NAME ?? 'text-embedding-v3',
  configuration: {
    baseUrl:
      process.env.OPENAI_BASE_URL ??
      'https://dashscope.aliyuncs.com/compatible-mode/v1',
  },
});

const milvusClient = new MilvusClient({
  address: MILVUS_ADDRESS,
});

/**
 * 重建 ES 索引并 bulk 写入
 */
asyncfunction seedElasticsearch(indexName, rows) {
  try {
    console.log(`\n[Elasticsearch]`);
    const client = new Client({ node: ES_NODE });

    const exists = await client.indices.exists({ index: indexName });
    if (exists) {
      console.log('删除已有索引...');
      await client.indices.delete({ index: indexName });
      console.log('✓ 已删除');
    }

    console.log('创建索引与 mapping...');
    await client.indices.create({
      index: indexName,
      mappings: {
        properties: {
          note_title: {
            type: 'text',
            analyzer: 'ik_max_word',
            search_analyzer: 'ik_smart',

```



```

    },
    note_body: {
      type: 'text',
      analyzer: 'ik_max_word',
      search_analyzer: 'ik_smart',
    },
    tags: { type: 'keyword' },
    mood: { type: 'keyword' },
    priority: { type: 'integer' },
    created_at: { type: 'date' },
    updated_at: { type: 'date' },
  },
},
});
console.log('✓ 索引创建成功');

const now = new Date().toISOString();
console.log(`写入 ${rows.length} 条文档...`);
await client.bulk({
  refresh: true,
  operations: rows.flatMap((row) => {
    const { id, ...rest } = row;
    return [
      { index: { _index: indexName, _id: id } },
      { ...rest, created_at: now, updated_at: now },
    ];
  }),
});
console.log('✓ ES 写入完成');
} catch (error) {
  console.error('Elasticsearch 出错:', error.message);
  throw error;
}
}

/**
 * 若集合已存在则删掉；创建集合、索引，加载后再插入向量数据
 */

asyncfunction seedMilvus(collectionName, rows, emb) {
  try {
    console.log(`\n[Milvus]`);

    const texts = rows.map((row) => `${row.note_title}\n${row.note_body}`);
    console.log('生成向量嵌入...');
    const vectors = await emb.embedDocuments(texts);
    const dim = vectors[0].length;
  }
}

```

```
const hasCollection = await milvusClient.hasCollection({
  collection_name: collectionName,
});
if (hasCollection.value) {
  console.log('删除已有集合...');
  await milvusClient.dropCollection({ collection_name: collectionName });
  console.log('✓ 已删除');
}

console.log('创建集合...');
await milvusClient.createCollection({
  collection_name: collectionName,
  fields: [
    { name: 'id', data_type: DataType.VarChar, max_length: 100 },
    {
      name: 'note_title',
      data_type: DataType.VarChar,
      max_length: 512,
    },
    {
      name: 'note_body',
      data_type: DataType.VarChar,
      max_length: 4096,
    },
    { name: 'mood', data_type: DataType.VarChar, max_length: 64 },
    {
      name: 'priority',
      data_type: DataType.VarChar,
      max_length: 16,
    },
    { name: 'tags', data_type: DataType.VarChar, max_length: 256 },
    {
      name: 'langchain_primaryid',
      data_type: DataType.Int64,
      is_primary_key: true,
      autoID: true,
    },
    {
      name: DOC_TEXT,
      data_type: DataType.VarChar,
      max_length: 10000,
    },
    {
      name: EMBEDDING,
      data_type: DataType.FloatVector,
      dim,
    },
  ],
});
```

```

    ],
  });
  console.log('✓ 集合创建成功');

  console.log('创建向量索引...');
  await milvusClient.createIndex({
    collection_name: collectionName,
    field_name: EMBEDDING,
    index_type: IndexType.HNSW,
    metric_type: MetricType.L2,
    params: { M: 8, efConstruction: 64 },
  });
  console.log('✓ 索引创建成功');

  try {
    await milvusClient.loadCollection({ collection_name: collectionName });
    console.log('✓ 集合已加载');
  } catch {
    console.log('✓ 集合已处于加载状态');
  }

  console.log(`插入 ${rows.length} 条...`);
  const insertData = rows.map((row, i) => ({
    id: row.id,
    note_title: row.note_title,
    note_body: row.note_body,
    mood: row.mood,
    priority: String(row.priority),
    tags: row.tags.join(','),
    [DOC_TEXT]: texts[i],
    [EMBEDDING]: vectors[i],
  }));

  const insertResult = await milvusClient.insert({
    collection_name: collectionName,
    data: insertData,
  });

  await milvusClient.flushSync({ collection_names: [collectionName] });

  const cnt = Number(insertResult.insert_cnt) || rows.length;
  console.log(`✓ Milvus 写入完成 (insert_cnt: ${cnt})`);
} catch (error) {
  console.error('Milvus 出错:', error.message);
  throw error;
}
}

```

```

/**
 * 主入口
 */
asyncfunction main() {
  try {
    console.log('\n连接 Milvus...');
    await milvusClient.connectPromise;
    console.log('✓ 已连接');

    await seedElasticsearch(INDEX_NAME, ROWS);
    await seedMilvus(INDEX_NAME, ROWS, embeddings);

  } catch (error) {
    console.error('\n错误:', error.message);
    console.error(error.stack);
    process.exit(1);
  }
}

main();

```

神光的编程秘籍

文档分别插入了 ES 的索引，Milvus 的集合

这样就分别可以走 es 的关键词检索、milvus 的语义检索了

我们让大模型把 query 重写成三个问题：

这样不同角度的问题可以召回会更多文档，然后 rerank 重排后选几个最相关的文档生成回答就好了。

创建 src/rag/query-augment.mjs

```
/**
 * 用大模型根据用户问题生成恰好 3 条不同角度的检索问句；每条问句各自走 ES / Milvus，最后合并去重。
 */
import { ChatPromptTemplate } from "@langchain/core/prompts";
import * as z from "zod";

export const QueryAugmentationSchema = z.object({
  queries: z
    .array(z.string())
    .length(3)
    .describe(
      "恰好 3 条中文检索问句：不同角度改写或扩写；保留订单号、品牌等字面信息；不要编造事实",
    ),
});

const AUGMENT_PROMPT = ChatPromptTemplate.fromMessages([
  [
    "system",
    `用户会给出一句中文问题。请另外写出恰好 3 条检索用的问句（与原意一致、角度尽量不同），便于搜索引擎或向
```

可改写说法、换提问角度、或略加限定词；专有名词、型号、订单号等必须保留原样。

只输出结构化字段 `queries`（长度为 3 的字符串数组）。`，

```
    ],  
    ["human", "{query}"],  
  ]);
```

```
function normalizeThreeQueries(original, list) {  
  const out = (list ?? [])  
    .map((s) => (typeof s === "string" ? s.trim() : ""))  
    .filter(Boolean);  
  while (out.length < 3) out.push(original);  
  return out.slice(0, 3);  
}
```

```
export async function augmentQuery(chatModel, query) {  
  const structured = chatModel.withStructuredOutput(QueryAugmentationSchema);  
  const chain = AUGMENT_PROMPT.pipe(structured);  
  try {  
    const raw = await chain.invoke({ query });  
    return { queries: normalizeThreeQueries(query, raw.queries) };  
  } catch {  
    return { queries: normalizeThreeQueries(query, []) };  
  }  
}
```

*/** 原始问题在前，其后接 LLM 生成的问句；不做去重，顺序固定；每条各跑一次 ES、Milvus */*

```
export function retrievalQueryStrings(original, augmentation) {  
  return [original, ...(augmentation?.queries ?? [])]  
    .map((s) => (typeof s === "string" ? s.trim() : ""))  
    .filter(Boolean);  
}
```

我们用 LLM 来改写问题生成三个不同角度的问题

然后实现多路召回 + 重排：

src/rag/hybrid-retrieval.mjs

```
/**  
 * 混合检索: LLM 重写为 3 条多角度问句 → 每条问句分别 ES + Milvus → 全量合并去重 → Rerank → LLM 作  
 * LangGraph: START → query_augment → es_recall // milvus_recall → merge → rerank → genera  
 */  
  
import "dotenv/config";  
import { Client } from "@elastic/elasticsearch";  
import { Document } from "@langchain/core/documents";
```

```

import { ChatPromptTemplate } from"@langchain/core/prompts";
import { Milvus } from"@langchain/community/vectorstores/milvus";
import { ChatOpenAI, OpenAIEmbeddings } from"@langchain/openai";
import { Annotation, END, START, StateGraph } from"@langchain/langgraph";
import { DashScopeRerank } from"../rerank/dashscope-rerank.mjs";
import {
  augmentQuery,
  retrievalQueryStrings,
} from"../query-augment.mjs";

const INDEX = "life_notes";

const HybridRetrievalState = Annotation.Root({
  query: Annotation(),
  queryAugmentation: Annotation(),
  esHits: Annotation(),
  milvusHits: Annotation(),
  merged: Annotation(),
  topDocuments: Annotation(),
  answer: Annotation(),
});

function docFromEsHit(hit) {
  const s = hit._source ?? {};
  const text = [s.note_title ?? s.title, s.note_body ?? s.content]
    .filter(Boolean)
    .join("\n");
  returnnew Document({
    pageContent: text,
    metadata: { id: hit._id, source: "es", ...s },
  });
}

/** ES 与 Milvus 结果拼接后仅按 metadata.id 去重, 保留首次出现 (通常 ES 在前) */
function merge(esDocs, milvusDocs) {
  const combined = [...(esDocs ?? []), ...(milvusDocs ?? [])].filter(
    (d) => d?.pageContent,
  );
  return dedupeDocsById(combined);
}

/** 去重键仅为 metadata.id (trim 后非空); 无 id 丢弃, 不按正文去重; 保留首次出现顺序 */
function dedupeDocsById(docs) {
  const seen = newSet();
  const out = [];
  for (const d of docs ?? []) {
    if (!d?.pageContent) continue;

```

```

    const id =
      d.metadata?.id != null ? String(d.metadata.id).trim() : "";
    if (!id) continue;
    if (seen.has(id)) continue;
    seen.add(id);
    out.push(d);
  }
  return out;
}

function printDocs(label, docs) {
  console.log(`\n=== ${label} (${docs?.length ?? 0} 条) ===`);
  for (let i = 0; i < (docs ?? []).length; i++) {
    const d = docs[i];
    const preview = (d.pageContent ?? "").slice(0, 200).replace(/\n/g, " ");
    console.log(`[${i}] ${preview}${d.pageContent?.length > 200 ? "..." : ""}`);
    console.log(`    metadata:`, d.metadata ?? {});
  }
}

/** 打印 LLM 生成的多角度检索问句及逐条检索列表 */
function printQueryRewrite(original, augmentation) {
  const qs = augmentation?.queries ?? [];
  const forRetrieval = retrievalQueryStrings(original, augmentation);

  console.log(`\n--- 查询扩展 (LLM 生成 ${qs.length} 条检索问句) ---`);
  console.log("原始 query:", original ?? "");
  for (let i = 0; i < qs.length; i++) console.log(`  [${i + 1}] ${qs[i] ?? ""}`);
  console.log(
    `
    \n逐条 ES + Milvus (共 ${forRetrieval.length} 条检索串, 含原始问题) :`,
    );
  for (let i = 0; i < forRetrieval.length; i++) {
    console.log(`  [${i + 1}] ${forRetrieval[i] ?? ""}`);
  }
}

function stringifyMessageContent(content) {
  if (typeof content === "string") return content;
  if (!Array.isArray(content)) return String(content ?? "");
  return content
    .map((c) =>
      typeof c === "string" ? c : typeof c?.text === "string" ? c.text : "",
    )
    .join("");
}

function formatDocsAsContext(docs) {

```



```

return (docs ?? [])
  .map((d, i) => {
    const meta = d.metadata ?? {};
    const src = meta.source ?? "";
    const id = meta.id != null ? String(meta.id) : "";
    const head = id ? `[$${i + 1}] id=${id}${src ? ` source=${src}` : ""}` : `[$${i + 1}]`;
    return `${head}\n${d.pageContent ?? ""}`;
  })
  .join("\n\n---\n\n");
}

```

```
const ANSWER_PROMPT = ChatPromptTemplate.fromMessages([
```

```

  [
    "system",
    `你是阅读用户「生活笔记」知识库并作答的助手。

```

规则:

- 只根据下方「检索片段」推断答案；片段里没有的信息不要编造。
- 若片段不足以回答，明确说明「笔记里未提到」，并可给出一句保守建议。
- 回答简洁有条理，可使用简短列表；口吻自然中文。`，

```
],
```

```
[
```

```
  "human",
```

```
  `用户问题: {query}`

```

检索片段:

```
{context}`,
```

```
],
```

```
]);
```

```
const NO_CONTEXT_PROMPT = ChatPromptTemplate.fromMessages([
```

```
  [
```

```
    "system",
```

```
    `你是阅读用户「生活笔记」知识库并作答的助手。当前没有检索到任何片段。

```

请用一两句话说明无法从笔记中回答，并礼貌询问用户是否换个说法或补充关键词。`，

```
],
```

```
  ["human", "用户问题: {query}"],
```

```
]);
```

```
export function compileHybridRetrievalGraph(esClient, milvus, reranker, chatModel) {
```

```
  const ES_K = 15;
```

```
  const MILVUS_K = 15;
```

```
  return new StateGraph(HybridRetrievalState)
```

```
    .addNode("query_augment", async (state) => ({
```

```
      queryAugmentation: await augmentQuery(chatModel, state.query ?? ""),
```

```
    })))
```

```
    .addNode("es_recall", async (state) => {
```

```

const qs = retrievalQueryStrings(state.query, state.queryAugmentation);
const n = Math.max(1, qs.length);
const kEach = Math.max(2, Math.ceil(ES_K / n));
const batches = awaitPromise.all(
  qs.map((q) =>
    esClient.search({
      index: INDEX,
      size: kEach,
      query: {
        multi_match: {
          query: q,
          fields: ["note_title^2", "note_body", "title", "content"],
          type: "best_fields",
          analyzer: "ik_smart",
        },
      },
    },
  ),
);
const flat = batches.flatMap((res) =>
  (res.hits?.hits ?? []).map(docFromEsHit),
);
return { esHits: dedupeDocsById(flat) };
})

.addNode("milvus_recall", async (state) => {
  const qs = retrievalQueryStrings(state.query, state.queryAugmentation);
  const n = Math.max(1, qs.length);
  const kEach = Math.max(2, Math.ceil(MILVUS_K / n));
  const batches = awaitPromise.all(
    qs.map((q) => milvus.similaritySearch(q, kEach)),
  );
  const flat = batches.flat();
  return { milvusHits: dedupeDocsById(flat) };
})

.addNode("merge", async (state) => ({
  merged: merge(state.esHits, state.milvusHits),
}))

.addNode("rerank", async (state) => {
  const merged = state.merged ?? [];
  if (!merged.length) return { topDocuments: [] };
  const topDocuments = await reranker.compressDocuments(merged, state.query);
  return { topDocuments };
})

.addNode("generate_answer", async (state) => {
  const query = state.query ?? "";
  const docs = state.topDocuments ?? [];
  if (!docs.length) {

```

```

        const chain = NO_CONTEXT_PROMPT.pipe(chatModel);
        const msg = await chain.invoke({ query });
        return { answer: stringifyMessageContent(msg.content).trim() };
    }
    const chain = ANSWER_PROMPT.pipe(chatModel);
    const msg = await chain.invoke({
        query,
        context: formatDocsAsContext(docs),
    });
    return { answer: stringifyMessageContent(msg.content).trim() };
})
.addEdge(START, "query_augment")
.addEdge("query_augment", "es_recall")
.addEdge("query_augment", "milvus_recall")
.addEdge(["es_recall", "milvus_recall"], "merge")
.addEdge("merge", "rerank")
.addEdge("rerank", "generate_answer")
.addEdge("generate_answer", END)
.compile();
}

const esClient = new Client({ node: "http://localhost:9200" });
const embeddings = new OpenAIEmbeddings({
    model: "text-embedding-v3",
    apiKey: process.env.OPENAI_API_KEY,
    configuration: {
        baseUrl: "https://dashscope.aliyuncs.com/compatible-mode/v1",
    },
});

const milvus = await Milvus.fromExistingCollection(embeddings, {
    url: "http://localhost:19530",
    collectionName: INDEX,
    textField: "doc_text",
    vectorField: "embedding",
});

const reranker = new DashScopeRerank({
    apiKey: process.env.OPENAI_API_KEY,
    model: "qwen3-rerank",
    topN: 3,
    baseUrl:
        "https://dashscope.aliyuncs.com/api/v1/services/rerank/text-rerank/text-rerank",
});

const chatModel = new ChatOpenAI({
    model: process.env.LLM_MODEL_NAME ?? "qwen-turbo",
    apiKey: process.env.OPENAI_API_KEY,
    temperature: 0.2,

```

```

configuration: {
  baseUrl:
    process.env.OPENAI_BASE_URL
},
});

/** 示例用户 query (字符串列表) */
const SAMPLE_QUERIES = [
  // "P0-20250409-K9 滤芯订单",
  "家里无线老是断断续续的咋整啊",
  // "那个黑凉粉粉怎么冲不结块",
  // "明火炖太久汤汁又黏又涩，起锅前要如何处理才不腻",
];

const graph = compileHybridRetrievalGraph(esClient, milvus, reranker, chatModel);

const drawable = await graph.getGraphAsync();
console.log(drawable.drawMermaid());
console.log();

for (const query of SAMPLE_QUERIES) {
  console.log(`query: ${query}`);

  const state = await graph.invoke({ query });

  printQueryRewrite(state.query, state.queryAugmentation);
  console.log("\n (原始 JSON) ", JSON.stringify(state.queryAugmentation));

  printDocs("Elasticsearch 检索", state.esHits);
  printDocs("Milvus 检索", state.milvusHits);
  printDocs("重排后保留", state.topDocuments ?? []);

  console.log("\n=== 大模型生成回答 ===\n");
  console.log(state.answer ?? "");
}

```

神光的编程秘籍

问题经过改写后，会生成 3 个不同角度的问题。

我们把每个问题，分别同时走 ES 关键词检索、Milvus 语义检索；

再把所有检索到的文档统一合并、去重，通过 Rerank 重排模型，筛选出和用户原始问题最相关的 3 篇文档，用来给到大模型生成最终回答。

到这里，我们的 RAG 就正式进化成 **多问题改写 + 混合多路检索 + 重排精筛** 的完整架构，也是企业级落地标准的完善版 RAG 方案。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

这节我们学了混合检索 + rerank 重排模型。

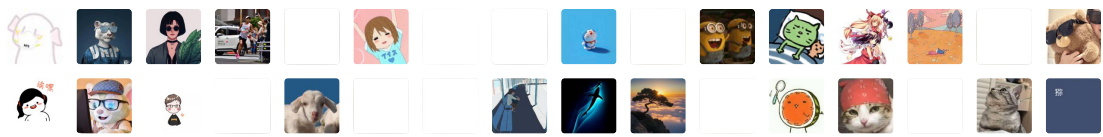
首先实现了用代码操作 es，包括创建索引、文档增删改查。

然后学了 rerank 模型，它和嵌入模型一样都是专用的模型，嵌入模型是文本转向量，重排模型是给文档和问题的相关性打分、排序。

之后基于 langgraph 实现了 query 改写、多路召回、去重、重排的 RAG 流程。

生产级的 RAG 一般都是这样：做问题重写，走关键词 + 语义的混合检索，最后用重排模型选出最相关的文档给大模型来生成回答。

2774 人付费



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

ElasticSearch 全文检索：倒排索引表 + IK 分词器 + BM25 算法

下一篇

Neo4j 知识图谱和 Graph RAG

修改于2026年5月3日

